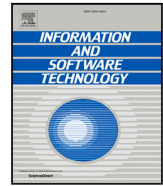




Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

Using AI-based coding assistants in practice: State of affairs, perceptions, and ways forward

Agnia Sergeyuk^{a,1}, Yaroslav Golubev^{a,*}, Timofey Bryksin^b, Iftekhar Ahmed^c^a JetBrains Research, Belgrade, Serbia^b JetBrains Research, Limassol, Cyprus^c University of California, Irvine, CA, United States

ARTICLE INFO

Keywords:

LLMs

AI assistants

Software development lifecycle

ABSTRACT

Context: The last several years saw the emergence of *AI assistants* for code — multi-purpose AI-based helpers in software engineering. As they become omnipresent in all aspects of software development, it becomes critical to understand their usage patterns.

Objective: We aim to better understand *how specifically* developers are using AI assistants, why they are *not* using them in certain parts of their development workflow, and what needs to be improved in the future.

Methods: In this work, we carried out a large-scale survey aimed at how AI assistants are used, focusing on specific software development activities and stages. We collected opinions of 481 programmers on five broad activities: (a) implementing new features, (b) writing tests, (c) bug triaging, (d) refactoring, and (e) writing natural-language artifacts, as well as their individual stages.

Results: Our results provide a novel comparison of different stages where AI assistants are used that is both comprehensive and detailed. It highlights specific activities that developers find less enjoyable and want to delegate to an AI assistant, *e.g.*, writing tests and natural-language artifacts. We also determine more granular stages where AI assistants are used, such as generating tests and generating docstrings, as well as less studied parts of the workflow, such as generating test data. Among the reasons for not using assistants, there are general aspects like trust and company policies, as well as more concrete issues like the lack of project-size context, which can be the focus of the future research.

Conclusion: The provided analysis highlights stages of software development that developers want to delegate and that are already popular for using AI assistants, which can be a good focus for features aimed to help developers right now. The main reasons for not using AI assistants can serve as a guideline for future work.

1. Introduction

In the ever-evolving landscape of technology, the symbiotic relationship between humans and Artificial Intelligence (AI) has ushered in a new era of innovation, transcending traditional boundaries and catalyzing unprecedented advancements. The recent advent of Large Language Models (LLMs) is one such advancement that, like many other fields, is significantly impacting software development [1].

LLM-powered AI assistants, such as GitHub Copilot [2], JetBrains AI Assistant [3], and Visual Studio IntelliCode [4], have emerged as invaluable assets. They assist developers in code generation [5], bug fixing [6], refactoring [7], testing [8], and almost every aspect of the software development life-cycle. Being trained on vast corpora of data,

these assistants provide intelligent suggestions and recommendations, thereby augmenting the capabilities of software developers and rapidly gaining popularity among them [9].

Given their rapid growth, understanding software developers' perceptions and needs regarding AI assistants is paramount in shaping the future of software development. Recent studies have started looking into why AI assistants are used, what their high-level drawbacks are [10–12], etc. However, the existing studies do not consider individual activities within the software development life-cycle and their specificities. Because of this, it is challenging to transform the results of such works into actionable implications, since they only deal with general concerns like quality and security, without taking into account

* Corresponding author.

E-mail addresses: agnia.sergeyuk@jetbrains.com (A. Sergeyuk), yaroslav.golubev@jetbrains.com (Y. Golubev), timofey.bryksin@jetbrains.com (T. Bryksin), iftekha@uci.edu (I. Ahmed).¹ The first two authors contributed equally to this work.<https://doi.org/10.1016/j.infsof.2024.107610>

Received 10 July 2024; Received in revised form 21 September 2024; Accepted 23 October 2024

Available online 29 October 2024

0950-5849/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

their concrete manifestations in the developer's workflow. Conversely, other works focus on individual narrow stages, e.g., code completion, test generation, etc., and study them in detail. While they are more specific, it is difficult to compare and prioritize their results, because they do not take the broader picture into account. The quick development of AI assistants requires a comprehensive yet granular analysis that considers the full software development lifecycle and its detailed steps at the same time, thus allowing to find areas where AI assistants are already employed to focus on them in the short term and identify shortcomings to address them in the future. To the best of our knowledge, no study has yet been conducted on such an analysis.

To overcome the existing research gap, we carried out a large-scale survey to determine where developers need AI support and what kind of support they need. More specifically, we developed, piloted, and ran a survey that focused on answering the following three research questions:

- RQ1** What is the general usage and perception of AI assistants among programmers?
- RQ2** To what extent do programmers utilize AI assistance for specific activities in software development, and how do they perceive these activities?
- RQ3** What are the reasons for not using AI assistants, and therefore, what needs to be improved?

In particular, we studied the following five broad SE activities: (a) implementing new features, (b) writing tests, (c) bug triaging, (d) refactoring, and (e) writing natural-language artifacts, as well as their individual stages. Our survey included 38 main questions, of them 5 being open-ended, which allowed the participants to freely express their thoughts. The survey attracted 547 complete responses, and after careful filtering, we study 481 responses. Our sample is experienced and diverse, with almost half of the respondents having more than 10 years of professional experience covering all the major programming languages and types of developed software.

Overall, the contributions of this work are the following:

- **Usage patterns.** We find that 84.2% of the respondents occasionally or regularly use AI assistants, that *Implementing new features* is the most popular activity to use assistants, and that among individual stages, generating and summarizing code are the most widely used.
- **Areas of focus.** Taking into account which activities the developers find less enjoyable and want to delegate, as well as what stages AI assistants are used on, we highlight areas where the research needs to focus on to bring value to users right now. This includes *Writing tests* in general and *generating data and resources for tests* in particular, as well as *Writing natural language artifacts*.
- **Reasons for not using AI.** We provide 20 distinct themes that represent different reasons why developers are not using AI assistants and report the main ones for all activities. The most popular ones are *Lack of need for AI assistance*, *AI-generated output being inaccurate*, *User's lack of trust and the desire to feel in control*, and *Lack of understanding context by AI assistant*.
- **Areas of future improvement.** Based on the prevalence of different reasons, we formulate the main areas of future work that are needed to overcome the shortcomings that the respondents describe. This includes the improvement of base systems, better integration into developers' workflow, and more actively educating and informing users about the assistants' capabilities and limitations.

The remainder of this paper is organized as follows. In Section 2, we discuss the related work. In Section 3, we describe how we developed and piloted our survey, how we found the participants, and how we analyzed the results. Section 4 presents these results in detail, and Section 5 describes their specific implications. Finally, we describe the potential threats to the validity of our study in Section 6 and conclude the paper in Section 7.

2. Related work

AI assistance is a very active area of research, and numerous studies have investigated its impact on how programmers code. In this section, we describe the main surveys and studies — both academic and industrial — that explore the users' perceptions and benefits from AI assistants.

Liang et al. [12] conducted an exploratory qualitative study on the usage of AI programming assistants, highlighting motivations for usage, usability challenges, and implications for creators and users of these tools. The study had 410 developers as participants. The authors concluded that while using AI to reduce keystrokes, finish programming tasks quickly, and recall syntax, developers sometimes struggle to receive AI outputs that align with their requirements and expectations.

Wang et al. [13] present a study that aims to understand practitioners' expectations on code completion, compare them with existing research, and highlight the need for researchers to develop techniques that meet practitioners' demands. The methodology involves semi-structured interviews with 15 professionals and an exploratory survey with 599 professionals. The authors found that practitioners expect code completion tools to work for different granularities and scenarios. They also expect a tool to be accurate, display personalized completion, be available offline, and be relatively fast.

Ziegler et al. [14] discuss the use of neural code synthesis in software development and study perceived productivity by investigating whether usage measurements of developer interactions with GitHub Copilot can predict perceived productivity as reported by developers in a survey. The authors found that the acceptance rate of shown suggestions is a better predictor of perceived productivity than more specific metrics regarding the persistence of completions in the code over time. This suggests that the rate at which suggestions are accepted drives developers' perception of productivity.

Zhou et al. [11] aimed at understanding the issues and challenges developers face when using Copilot in practice, as well as their underlying causes and potential solutions. The results of the analysis of data from 476 GitHub issues, 706 GitHub discussions, and 142 Stack Overflow posts highlight that users commonly face Operation and Compatibility Issues with Copilot. The most frequent causes include Copilot Internal Issues, Network Connection Problems, and Editor/IDE Compatibility Issues. Predominant solutions involve Bug Fixes by Copilot, Configuration/Setting Modifications, and Using Suitable Versions. The authors suggest integrating a code explanation feature to enhance Copilot's overall utility and effectiveness in practical development scenarios, considering the additional time required for code suggestion verification.

Pothukuchi et al. [15] examine the potential of generative AI to automate and enhance traditional software development practices, thereby improving efficiency and reducing costs. A total of 30 professionals participated in the interview study and highlighted significant improvements in development speed and code quality. The authors indicate that generative AI can significantly transform the SDLC by automating repetitive tasks and providing intelligent code suggestions, and propose a new model called Generative AI-Assisted Software Development Lifecycle.

Mozannar et al. [16] conducted a comprehensive study to inform the understanding of how developers interact with AI tools and how to improve this experience. They studied the impact of GitHub Copilot on programmers' behavior during coding sessions. As a result, they identified 12 common programmer activities related to AI code completion systems. The researchers found that developers spend more time reviewing code than writing it.

Barke et al. [17] propose a grounded theory of the bimodal nature of programmers' interactions with AI assistance that states that there are two main interaction modes between which developers fluidly switch while programming — exploration and acceleration. In acceleration mode, the programmer already knows what they want to do next, and

AI helps them get there quicker; interactions in this mode are fast and do not break programmer's flow. In exploration mode, the programmer is not sure how to proceed and uses Copilot to explore their options or get a starting point for the solution; interactions in this mode are slow and deliberate, and include explicit prompting and more extensive validation.

Moreover, several companies from industry conducted large-scale studies to form an understanding of the market of AI tooling for coding. A survey of 89,184 developers conducted in 2023 by Stack Overflow [9] found that 70% are using or planning to use AI tools, with 77% expressing a favorable view. Respondents noted increased productivity with these tools, as well as their trust in their accuracy. The most recent Stack Overflow survey [18], conducted on 1700 people, revealed the continuation of the trend — users report more quality work time.

A GitHub survey [19] of 500 non-manager developers found that 67% have used AI tools both at work and in personal projects. 70% believe AI coding tools will benefit their work, primarily for upskilling and productivity gains. 81% expect these tools to enhance team collaboration, particularly in security reviews, planning, and pair programming.

The JetBrains Developer Ecosystem Survey [20] found that developers are optimistic about AI advancements and actively use its capabilities despite security and ethical concerns. 59% have security concerns, 42% ethical concerns, yet 53% are willing to use generative AI services for work. Most commonly, developers use AI to ask general software development questions and generate code, comments, or documentation.

In general, existing research on the utilization of AI assistants covers a range of perspectives, with some studies focusing on a single activity, such as code completion, while others offer more generalized insights that may overlook the nuances of different specific activities within software development. In contrast to this existing body of work, our study aims to combine the strengths of both approaches to allow for a more direct comparison of various areas of application of AI assistants. Taking into account the entire software development lifecycle, while also asking about specific low-level work aspects, can allow our findings to be both concrete and comprehensive. Having a wide and detailed palette of opinions within one large-scale survey provides an opportunity to prioritize both short-term improvements to features that are already in use and long-term research into reasons why some features are not yet widely employed.

3. Methodology

Our survey was aimed at answering the following three research questions:

- RQ1** What is the general usage and perception of AI assistants among programmers?
- RQ2** To what extent do programmers utilize AI assistance for specific activities in software development, and how do they perceive these activities?
- RQ3** What are the reasons for not using AI assistants, and therefore, what needs to be improved?

In this section, we describe our survey, the way we attracted our participants, and the analysis we carried out on the gathered data.

3.1. Survey structure

The full survey, including all the stages of activities, is available in our supplementary materials [21].

Firstly, we ask the developers an array of general demographic questions (years of professional experience, employment status) and questions about their jobs (primary programming languages, job role, job level, and type of developed software). Next, to answer **RQ1** and establish the general usage and perception of AI assistants, we

inquire about the participants' familiarity with specific popular AI assistant tools, as well as their opinions about the main qualities of the code generated by AI assistants: usability, accuracy, alignment with non-functional requirements, and security.

The remaining survey is divided into five parts that correspond to five large divisions of software activity in research and in practice [22], which includes (1) implementing new features, (2) writing tests, (3) bug triaging, (4) refactoring, and (5) writing natural language artifacts. Within each of these blocks, the questions follow the same pattern.

First, to answer **RQ2** and discover how participants perceive the activities, we ask them to rate each activity on a 5-point Likert scale in three dimensions:

1. From *Unpleasant* to *Enjoyable*.
2. From *Difficult* to *Easy*.
3. From *Would do it myself* to *Would delegate to an AI assistant*.

Then, we deepen our understanding of activities by considering their individual *stages*, which allows us to make the results of this research not only comprehensive but also detailed. For each activity, we provide a list of *stages*, or component steps, within this activity and ask the participant whether they employ an AI assistant on this stage. Some stages are the same for all activities, for example, "*Chatting with an AI assistant to brainstorm ideas*", while the majority are unique, like "*Generating data and resources for tests (e.g., inputs and outputs)*" for writing tests, or "*Generating commit messages*" for writing natural-language artifacts. Also, the participants could enter their own steps in the "*Other*" field.

Finally, to answer **RQ3**, for each activity, we ask an open-ended question about what prevents the participants from using AI assistants in the stages they did not select and whether they would use an AI assistant in those steps if the issues were resolved. Responses to these questions can allow us to not only quantitatively determine the least popular stages, but also establish distinct reasons for not using assistants there, thus leading the way to fixing them.

To ensure the robustness of our survey, we consulted experts in survey design and carried out five pilot runs with our colleagues. The experts included members of the Surveys Team and the Research Core team at JetBrains, who specialize in designing surveys and other forms of quantitative and qualitative research. The list of pilot participants was also diverse — two software developers, two ML engineers, and one UX researcher. Overall, the feedback from the experts and the pilot participants was used to update and refine the survey. In particular, we compiled and polished the list of stages for each activity. Also, the participants highlighted the need to have "*Chatting with an AI assistant to brainstorm*" as an option for each activity, as well as having an explicit "*None*" option.

3.2. Data collection

The participants were invited from both Industry and Academia communities. The survey link was advertised via personal social media by the authors of the paper, three of whom are the employees of JetBrains Research, and was emailed to the list of people curated by JetBrains who gave their consent for contact with research purposes. Importantly, not all people in this mailing list are customers of JetBrains products, and the list includes the users of various IDEs and AI assistants. As a thank you, participants were given the opportunity to enter a draw for one of five 50 USD Amazon eGift Cards or an equivalent-value JetBrains product pack. The study was conducted in line with the company's ethical standards, adhering to the values and guidelines outlined in the ICC/ESOMAR International Code [23].

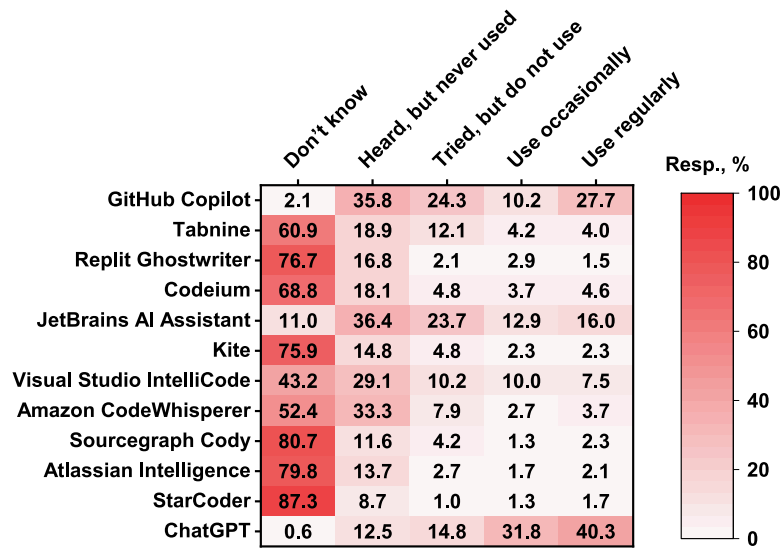


Fig. 1. The tools used by the respondents.

3.3. Data analysis

After collecting the responses, we analyzed them. This includes filtering the results, calculating correlations, and processing open-ended questions.

Filtering results. At the end of the three weeks during which the survey remained open, we received 780 responses, of which 547 were complete. Among these 547 responses, 542 came from the mailing list, indicating a strong industrial backing of our sample. This allowed us to collect diverse opinions from experienced developers acquainted with a variety of different tools, as evidenced further in Sections 4.1 and 4.2.

To ensure the quality of our data, we further applied two checks. Firstly, research shows that responses that took too little or too long time to complete are less reliable [24]. To filter them out, we followed an established practice of filtering out outliers, applying Tukey's fences [25]. Because our survey was designed to facilitate quick responses and not fatigue the participant, most of the responses are relatively quick (the median time of all complete responses is 11 min), so this filtering did not affect quick responses. However, it did filter out 66 responses that took 30 or more minutes to complete, with some of them taking several hours. Additionally, we checked our results for responses where the first option is selected for each question [24]. We did not find such cases. Thus, the final data consisted of 481 responses.

Calculating correlations (RQ2). For each activity, we calculate correlations for Likert-scale values of the enjoyability, easiness, and delegatability of the activity. For all correlations, we used the Spearman correlation coefficient, which is recommended to be used for Likert scales [26].

Processing open-ended questions (RQ3). For questions about why the respondents do not use AI assistants, we used thematic analysis [27] to group the responses. The first stage of the analysis was *open-coding the responses*. We decided to process all the responses together, going activity by activity: firstly, *implementing new features*, then *writing tests*, etc. This way, the obtained codes and later themes would be general enough, however, activity-specific codes would not be lost. The first two authors independently read all the responses and independently coded them. Then, the two authors discussed their codes until they reached a consensus for each response using a negotiated agreement approach [28].

From 481 respondents in five open-ended questions, we got 1764 responses. After filtering out responses such as "NA", "Nothing", "Don't know", responses that were written in languages other than English,

and responses that did not provide any clear criticism or reason, 1559 responses were left and resulted in 147 different codes.

The second stage of our analysis was *merging the codes into themes*. This task was performed independently by the same first two authors. Following the established methodology [29], both authors iteratively merged similar codes into themes. Then, the authors discussed their themes and the position of each code within them until they reached a consensus. This process resulted in a total of 20 distinct themes. You can find the list of themes and their corresponding codes in our supplementary materials [21].

4. Results

4.1. Demographics

Before moving to the results themselves, let us briefly showcase the demographics of our studied sample. An extended list of the answers to all the demographic questions can be found in supplementary materials [21].

The respondents came from 71 different countries and territories from all the continents. They use more than 30 diverse languages, with the most popular being *Python* (45.5% of respondents), *JavaScript* (31.6%), and *Java* (26.4%). They also develop diverse types of software, including *Websites* (51.1%), *Utilities* (36.4%), and *Libraries/Frameworks* (30.6%).

Our sample consists mostly of experienced programmers, with 48.9% having more than 10 years of experience and 28.9% having more than 15. 74.4% of our sample are fully employed in corporations and organizations, with some more being partially employed, self-employed, or freelancing. The majority (86.7%) identify themselves as *Software developers*, but the sample also contains some of the other major technical positions: *DevOps engineers*, *Architects*, *Team leads*, etc.

4.2. RQ1: General usage and perception of AI assistants in SE

Our first RQ focused on the current state and perceptions of using AI assistants in software development.

Used tools. The heatmap in Fig. 1 shows specific tools the respondents use in their work. Overall, 84.2% of the respondents mentioned that they use at least *some* tool occasionally or regularly. In terms of specifics, we can see *ChatGPT* (72.1%), *GitHub Copilot* (37.9%), *JetBrains AI Assistant* (28.9%), and *Visual Studio IntelliCode* (17.5%) are

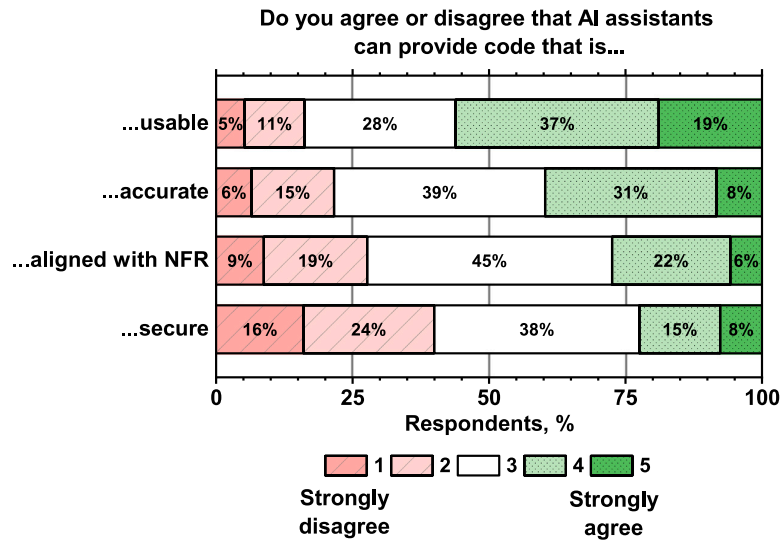


Fig. 2. General opinions of respondents about code provided by AI assistants. NFR stands for non-functional requirements.

the most frequently used tools. As for the other tools, more than half of the respondents do not know about them. This demonstrates that the field of AI assistants is dominated by just several big players and smaller ones are used more rarely. In the write-in responses, the most popular one was *Google Gemini/Bard* (6.4% of respondents), with some others mentioned being *Ollama* and *Perplexity*.

Opinions about AI-generated code. Next, Fig. 2 demonstrates the general opinions of the respondents about the code provided by AI assistants. We can see that the quality most positively rated is *usable*, with 56% of respondents agreeing or strongly agreeing with it. 39% of respondents agree or strongly agree that the code provided by AI is *accurate*, while 39% neither agree nor disagree with this statement. Regarding the *alignment with non-functional requirements*, 28% of respondents are positive, and 45% are neutral. Finally, the most negative opinion is shared about the code being *secure*, with just 23% of respondents agreeing and as many as 40% of them disagreeing to some extent.

We also separately checked these opinions from the part of our sample who do not use AI assistants (15.8% of respondents who did not select using any tool occasionally or regularly). While they are a minority, their opinion is much more negative: even for the most positive quality of *accurate*, only 21% of them agree, and 45% disagree, whereas for the other three qualities, more than half disagree. This might indicate that increasing the adoption of AI assistants requires overcoming the corresponding issues and convincing developers.

Takeaway 1: The overall opinion regarding code provided by AI assistants differs, with the most positive aspect being its usability and the most negative being its security.

4.3. RQ2: Activities and their stages

Our second RQ focused on how exactly AI assistants are used in specific SE activities and specific stages within them.

Activities. Firstly, we compile and compare the more general opinions about the activities themselves to highlight the context of the importance of AI assistance in those activities. For each of them, we asked the participants (1) how unpleasant or enjoyable it is, (2) how difficult or easy it is, (3) how likely it is that they would delegate it to the AI assistant, in the form of Likert scales. The results are presented in Fig. 3. We can see some noteworthy differences between the activities.

Implementing new features is the most enjoyable activity, with as many as 86% of respondents rating it positively. It is also the least likely

Table 1

Spearman correlations between answers to questions about Enjoyability, Easiness, and Delegability of each of the five main activities. The underlined values are statistically significant with $p < 0.01$.

Activity	Enj./Del.	Eas./Del.
New features	−0.04	<u>0.12</u>
Writing tests	−0.24	−0.08
Bug triaging	−0.20	−0.08
Refactoring	−0.14	0.09
NL artifacts	−0.32	0.01

activity to be delegated to an AI assistant, with 48% of respondents answering negatively. On the other hand, *writing tests* and *writing natural language artifacts* are both noticeably the least enjoyable of activities (46% negative scores for tests and 43% negative scores for artifacts), and they are the ones that the developers want to delegate the most — 70% and 66%, respectively.

Refactoring is the second most enjoyable activity, with 64% positive assessment. Nonetheless, 49% of the participants would delegate that activity to an AI assistant. People hold more mixed opinions regarding *bug triaging* — 32% of respondents marked it as unpleasant and 36% as enjoyable, with 34% unwilling and 46% willing to delegate it to AI.

We performed a Spearman correlation [26] to examine the relationship between enjoyment and willingness to delegate the activity. Presented in the first column of Table 1, the results reveal a negative correlation between these aspects, indicating that if a person enjoys the given activity less, they are more likely to delegate it. Notably, writing natural artifacts shows a moderately negative correlation ($r = -0.32$, $p < .01$).

Additionally, we explored the correlation between the difficulty of the activity and the willingness to delegate it. Overall, from Fig. 2, it can be seen that opinions about the easiness of activities are more uniform, without as big differences as for other qualities. The results, presented in Table 1 support the idea that the correlation between the difficulty of the activity and willingness to delegate is weak and mostly not statistically significant, meaning that there is no connection between these two aspects of activity for our sample.

Takeaway 2: Among the main activities studied, implementing new features is the most enjoyable and the least likely to be delegated to an assistant, while writing tests and writing natural-language artifacts are the most unpleasant and the most likely to be delegated.

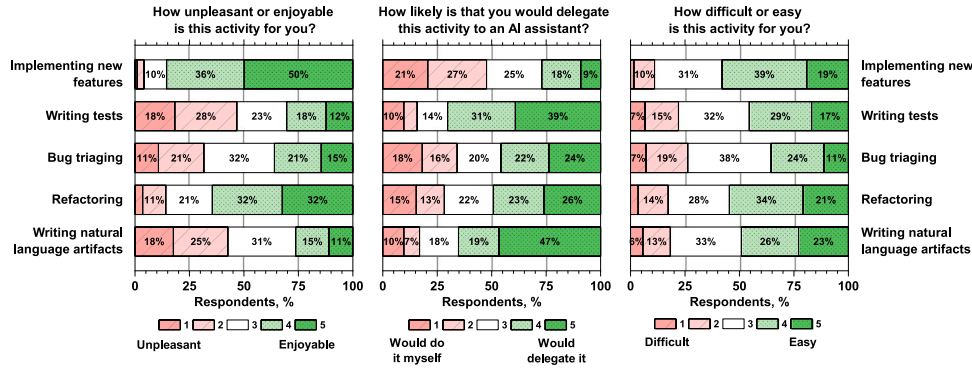


Fig. 3. The general opinions of respondents about each of the five main activities.

Now, let us delve into each of the activities and the specific stages within them. The comprehensive results for all stages of all activities can be found in Fig. 4.

Stages of Implementing new features. The first thing we can notice about *Implementing new features* is that only 12.7% of respondents — the lowest percentage among all activities — chose the option *None of the above*, meaning that they do not utilize AI tools for any of the stages in this activity. This means that the other 87.3% use AI in at least one of the stages of this activity, making it the most popular one for AI assistance. Interestingly, in Fig. 3 we have seen that the smallest percentage of developers wanted to delegate *Implementing new features*, yet in reality the largest percentage of them use AI Assistants for some of the aspects of this activity. This can be explained by the fact that developers assume *full delegation* that they do not want, but actually employ assistants for some stages.

Chatting with AI assistant to brainstorm ideas, which is present as a stage in all activities, has the noticeably highest percentage in this one — 57% of our respondents chose this option. This indicates the situational importance of conversational functionality for AI assistants.

The other two popular stages of *Implementing new features* are *Generating the new code* (57.4%) and *Exploring the APIs of unknown libraries* (50.3%). Interestingly, all the top-3 most popular choices represent different kinds of assistance, indicating the crucial nature of assistants' versatility.

Among the less popular choices are *Inserting the code into the codebase* (25.8%) and *Identifying where to insert the new feature by using code comprehension* (20%), potentially indicating that developers are still more comfortable with making important decisions and changing their code manually.

Stages of Writing tests. Here, we can see that 24.7% of all participants selected the option *None of the above*, meaning that three-quarters use AI assistants in this activity.

The most popular stage is *Generating tests*, with as many as 60.7% of the respondents selecting it. Another popular stage is *Generating data and resources for tests*, which was also selected by more than half of respondents — 54.7%. This makes those two stages good cases for current improvement.

The least selected option is *Finding untested areas of code*, with only 32% selection rate, once more highlighting that people are less willing to use AI in stages requiring precision.

Stages of Bug triaging. The first thing that is noticeable from the distribution of respondents for *bug triaging* is the highest percentage of those selecting *None of the above*. As many as 34.5% of the respondents do not use AI assistants in any way in this activity.

Another noticeable result is that stages are more equally distributed for bug triaging. The two most popular stages here are *Using code comprehension to summarize the most likely causes of the bug* (43.7%) and *Generating the code for a potential bug fix* (42.4%).

Again, one can notice that *Applying the bug fix to code* is among the least popular responses (27.9%), indicating that here too developers prefer to manually change their code.

Stages of Refactoring. Similarly to *Writing tests*, for *Refactoring*, about a quarter of respondents chose *None of the above*, indicating that three-quarters of them use AI tools at least in some stages.

The most popular response, with 56.1% of respondents, is *Generating the refactored version of the code*. Another very popular response is *Using code comprehension to explain which specific refactoring is necessary and why*, with 49.3% of respondents selecting this option. Like the popular code comprehension option in *bug triaging*, this shows the power of the explanatory features of AI assistants.

Interestingly, though, the choice of *Applying the refactoring to code directly* is more popular (38.7%) than analogous choices for *Implementing new features* and *Bug triaging*. This might indicate a larger trust in AI when it comes to refactoring or a less critical nature of carrying out refactoring.

Overall, it can be noticed that for *Refactoring*, a lot of options have a high percentage of respondents, indicating the general importance of AI assistance in this activity.

Stages of Writing natural-language artifacts. Similarly to *Writing tests* and *Refactoring*, here also about a quarter of respondents chose *None of the above*, and three-quarters of them use AI assistants in at least one stage.

The two options with the highest percentage of responses — 51.1% — are *Generating comments at specific points in the code* and *Summarizing code into docstrings*. Both of these responses relate to in-code comments, indicating that among code-related natural language artifacts, they are the ones for which people use AI assistants the most.

The next most common response is *Summarizing the architectural description of the codebase* (46.8%), again indicating the importance of the explanatory features of AI assistants. It can be noticed, however, that other artifacts—*Larger textual descriptions (such as READMEs)* and *Commit messages*—while being different from regular comments, are also relatively popular to be generated, with 40.3% and 38.7% of respondents, respectively.

Takeaway 3. *Generating and summarizing features of AI assistants are usually the most popular. Directly applying the new code to the codebase and searching for places to apply it are less prevalent, although still used by a non-negligible percentage of respondents.*

4.4. RQ3: Reasons for not using assistants

Finally, it is critical to understand why developers are hesitant to use AI assistants for some development activities. To do that, we studied their responses to open-ended questions about factors that prevent respondents from using AI assistants. Our thematic analysis resulted in 20 distinct themes. Their full list with detailed description

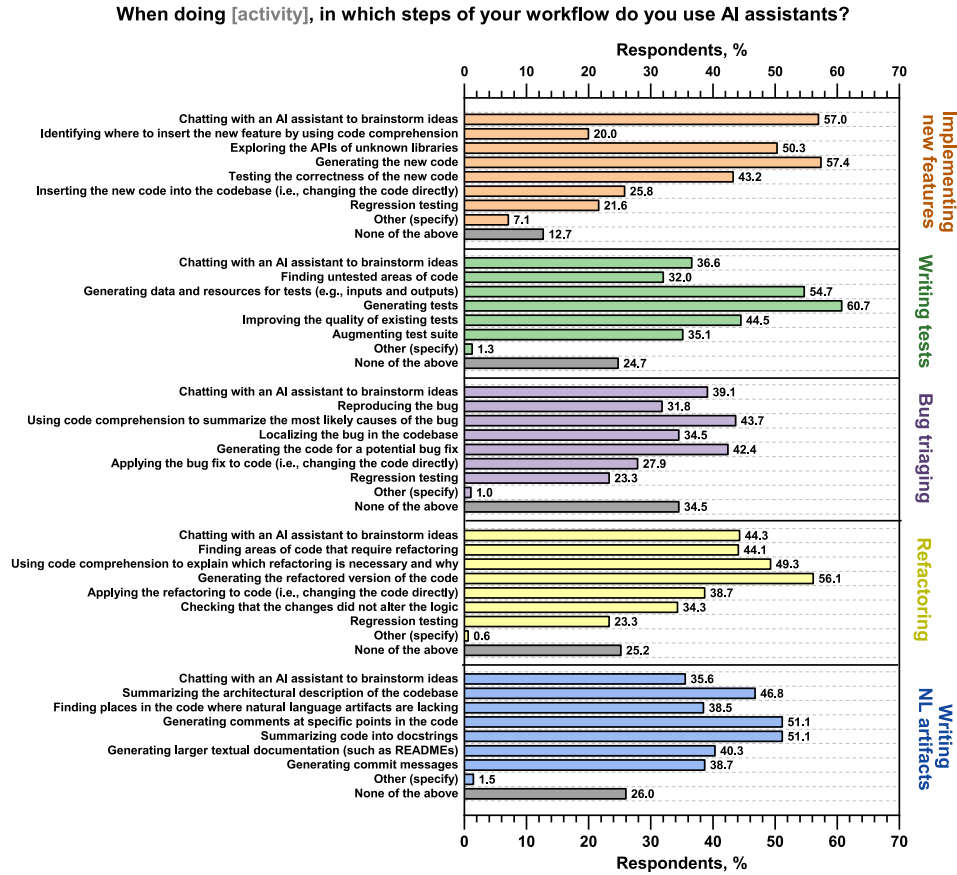


Fig. 4. The percentage of respondents who use AI assistants in different stages of different activities.

is provided in Table 2, with the percentage calculated from all 1559 coded responses across all five activities. We first discuss the themes in general and then highlight more specific problems for each activity.

General themes. The most prevalent theme is the *Lack of need for AI assistance*, mentioned in 22.5% of all responses. The respondents highlighted that they did not try assistants for some stages, and sometimes directly mentioned that they do not think the assistance is necessary because they can do the task themselves or they do not perform this task at all. Moreover, sometimes, they mentioned that the existing non-AI-based tools already work well.

The second most popular theme is *AI-generated output is inaccurate*, highlighted by 17.7% of responses. This group of issues relates to direct inaccuracies of AI-generated output, incorrect code, hallucinations, etc.

Next is *User's lack of trust and the desire to feel in control*, with 15.7% of responses. Respondents often mentioned not trusting AI-based tools, the results being unreliable, and their desire to maintain control over the project. This theme is largely prevalent in responses about applying the changes directly to code, confirming our notion from RQ2 that a lot of respondents are not comfortable with this. P380 wrote: *"I wouldn't fully trust the AI to apply a fix and ship it to production without supervision"*.

14.4% of responses highlighted the theme of *Lack of understanding context by AI assistants*. Sometimes, this relates to more technical and specific things, in particular, the inability of an AI assistant to analyze the full codebase or gain access to third-party code or company-specific artifacts. Some respondents, however, mentioned a broader lack of understanding: understanding the requirements of the task, business logic, etc.

Other, less prevalent themes include *User's desire to perform the task themselves and learn*, *Company policies*, *NDAs*, etc., *Workflow disruption*, and others.

In the following, we highlight specific issues mentioned in different activities. For each activity, we report the three themes, the relative percentage of which in that activity's responses is the highest compared to the overall percentage presented in Table 2, together with the reasoning that we collected from the responses.

Specifics of Implementing new features. *AI-generated output being inaccurate* was mentioned in 28.4% of all coded responses to the question about implementing new features. Here, incorrect code and hallucinations are especially critical, therefore, people encountering them are cautious in using AI for this activity. Some respondents mentioned that the AI assistant's output requires careful editing and that the errors are sometimes subtle and difficult to fix. P768 wrote: *"In my experience, AI's outputs very often contain subtle errors, which is why I have become very cautious"*.

Another aspect important for new features is *Time inefficiency*, raised in 11.3% of answers for this activity. Some respondents mentioned that correctly prompting the AI assistant takes longer than writing the code themselves, and some mentioned that fixing the generated code takes longer.

Workflow disruption was mentioned in 3.5% of responses regarding *Implementing new features*, with respondents not being used to utilizing assistants or mentioning that they interrupt their flow. P245 explained this in detail: *"... In my head I am several steps ahead of the code which is currently being entered into the file, and the AI assistants force me to backtrack constantly to analyze whether what they wrote was what I actually meant to write"*.

Specifics of Writing tests. For *Writing tests*, as many as 32.1% responses mentioned *Lack of need for AI assistance*. Some respondents do not do testing at all, and some mentioned that the existing tools work well for finding untested areas of code. P98 wrote: *"There are multiple*

Table 2

The resulting themes in the answers to the question “What prevents you from using AI assistants in the workflow steps you didn’t select above?”.

Theme	Description	Responses ^a
Lack of need for AI assistance	Respondents do not try the assistants, express no need or no interest in them, state that the existing non-AI-based tools work well, etc.	22.5%
AI-generated output is inaccurate	The output generated by an AI assistant is inaccurate, the code is incorrect, the model hallucinates, etc.	17.7%
User’s lack of trust and the desire to feel in control	Respondents do not trust the model, consider it to be unreliable, want to feel control over their code, believe that certain things have to be done by people, etc.	15.7%
Lack of understanding context by AI assistant	The assistant does not understand the context of the task or the underlying reason, cannot analyze the full codebase, cannot understand the requirements or business logic, etc.	14.4%
User’s limited knowledge or understanding of AI assistants and their capabilities	Respondents do not know enough about the AI assistants or how they work, do not know how to do the given task with it, do not know what to ask, etc.	10.2%
User’s desire to perform the task themselves and learn	Respondents want to do the work themselves, love the process, want to understand the code better or to learn, etc.	7.3%
Time inefficiency	Using AI assistants does not save any time for the respondent, it takes longer to prompt the assistant or to later fix the output than to do the task themselves, etc.	5.5%
AI-generated output is not useful	The output generated by an AI assistant is not useful, rudimentary, etc.	4.6%
Company policies, NDAs, etc.	Respondents are prohibited from using the AI assistants by their companies, NDAs, various policies, etc.	3.6%
User’s negative attitude towards AI	Respondents feel general negative emotions towards AI: hate, not wanting to rely too much, fear of being replaced, etc.	3.2%
Lack of compliance with non-functional requirements in AI assistant’s output	The output generated by an AI assistant does not fit the styling conventions of the project, has poor readability, is too long or too short, is not human-like, etc.	3.2%
Legal and ethical considerations	Respondents are worried about their privacy, about the copyright of the output, think that using AI is unethical, etc.	3.1%
Challenges with AI integration and usability in development environments	The AI assistant is not integrated well into the IDE, does not have the necessary functionality, has tooling and usability problems, etc.	3.0%
Lack of access	Respondents cannot access AI assistants, they are too expensive, etc.	2.6%
Security concerns	Respondents are worried about security, the AI assistants providing unsafe code, etc.	2.3%
Limitations in AI’s creativity	AI assistant lacks creativity, cannot provide complex solutions, provides generic code, etc.	2.2%
Workflow disruption	Respondents are not used to using AI assistants, have challenges with adopting it, feel like it breaks their workflow, etc.	2.1%
Limitations of knowledge in the AI model	The model in the AI assistant is outdated, lacks domain knowledge, is not trained well for non-English language, etc.	1.9%
Inefficiency for specific tasks	The AI assistant is incapable of carrying out some specific task, etc.	1.7%
Challenges in communicating user intentions to AI assistant	Respondents find it hard to communicate their intentions to the AI assistant, the assistant cannot follow instructions, requires directly pointing it to the necessary code, etc.	1.0%

^a The percentage is calculated from all 1559 coded responses across all activities.

extensions for both Rider and Visual Studio as well as commandline tools that show code coverage, so there is no need to use AI for that”.

Also, 7.8% of responses about tests highlighted that *AI-generated output is not useful*, and 2.9% of responses mentioned *Limitations in AI’s creativity*. Specifically, respondents mentioned that the generated tests are sometimes useless, rudimentary, and that the assistants cannot provide complex solutions. P525 wrote: “The generated code is not relevant most of the time, . . . , It is suitable for elementary tests only”.

Specifics of Bug triaging. 18.5% of responses related to *Bug triaging* mentioned *Lack of understanding context by AI assistant*. Respondents mentioned that finding and fixing non-trivial bugs requires a large context, both in terms of code and in terms of a more high-level, conceptual understanding of the program, which the AI assistants lack. P253 wrote: “A bug can come from a subpar execution of an idea or because a previously unknown behavior of the library/programming language/api. Investigating and fixing such bugs requires understanding a great deal of information.”.

Also, 3.1% of responses in *Bug triaging* mention *Inefficiency for specific tasks*, specifically mentioning that AI assistants are not capable of finding bugs or fixing them.

Finally, *Bug triaging* is the main source for “Challenges in communicating user intentions to AI assistant”, with 2.5% of responses mentioning it. P128 wrote: “AI assistants can’t understand all the details why there might be a bug because you can’t describe it (otherwise you would know the problem)”.

Specifics of Refactoring. For *Refactoring*, 21.9% of responses mentioned *User’s lack of trust and the desire to feel in control*. This is important for refactoring because it is crucial for the refactored code to not alter the logic of the original code. To this point, P60 wrote: “I prefer to make sure myself that the new refactored code does not change the logic”.

Also, as many as 10.4% of responses mention *User’s desire to perform the task themselves and learn*. Some respondents use refactoring as an opportunity to understand the code better and some just enjoy this activity overall. P558 mentioned this in their response: “Refactoring is one of the most enjoyable aspects of coding. It not only improves the readability and reusability of code but helps better understand it”.

4% of responses in this activity also mention *Lack of compliance with non-functional requirements in AI assistant’s output*, highlighting that the code refactored by the AI assistant may lose readability and not adhere

to the project's preferences. P47 wrote strongly: "...Auto-generated code is usually total garbage when it comes to structure and formatting".

Specifics of Writing natural-language artifacts. 29% of responses about natural-language artifacts contained the theme of *Lack of need for AI assistance*. Interestingly, Section 4.3 and Fig. 3 indicated that developers strongly want to delegate this activity. However, a lot of respondents said that they did not try AI assistants for certain artifacts, especially larger ones like READMEs. Moreover, some of them indicated that the existing tools work fine for them.

A crucial theme for natural artifacts is *Lack of compliance with non-functional requirements in AI assistant's output* (9.5% responses). Respondents mentioned that the text provided by the assistant does not sound human-like, may be bloated, verbose, not fitting the conventions of the projects, or overall not useful to the human reader. P273 succinctly put it like this: "Language produced by LLMs is often unnatural and does not match the required tone and clarity".

Finally, 2.7% of responses about this activity mention *Workflow disruption*, with P356 writing simply and directly: "I'm not used to it yet".

Overcoming the drawbacks. At the very end of each block, we asked the participants whether they would consider using the AI assistant for the stages they did not select if it did not have the described drawbacks. The detailed figure can be found in our supplementary materials [21]. While *Writing tests* and *Writing natural-language artifacts* are still the most likely to be delegated (71% and 72%, respectively), all the other three activities are also over 60% positive, indicating that in principle, the respondents are open to using AI assistants if the shortcomings are addressed.

Takeaway 4. The top reasons why developers are not using AI assistants are the lack of need, AI-generated output being inaccurate, lack of trust, and the lack of understanding of context by the assistant.

5. Discussion & implications

The results of our survey can provide a user-centered approach to focus the future development of AI assistants. The homogeneous comparison of different activities and different stages within them can help us select the most promising and the most problematic aspects of assistance usage.

5.1. What needs our focus now

The results of our study support the existing notion about the current state of affairs of using AI assistants in software development as favorable and rapidly developing [12,14]. We see that people adopted the technology in their workflows and see the code provided by AI as usable and fairly accurate. However, in line with previous studies [30–32], we see that people are cautious regarding AI-generated code being insecure. We perceive this serious drawback as an opportunity for the community to research and develop solutions to provide users with more safe and secure technology.

Our high-level comparison of activities (Section 4.3) indicates that in terms of activities, *writing tests* and *writing natural language artifacts* are good candidates for improvement. The comparison of responses to different questions allows us to conclude that respondents find these tasks to be somewhat unpleasant and express their desire to have them automated. Our detailed subsequent questions about individual stages highlight how specifically respondents use AI assistants within these activities: *generating tests*, *generating data and resources for tests* (e.g. inputs and outputs), *summarizing code into docstring*, *generating comments at specific points in the code*. While some of these tasks are well-established, others are less studied — for example, using AI assistants for generating separate resources for tests requires more attention from the research community. Combining the established collaboration between developers and AI on these tasks with the willingness to delegate activities to

AI, they warrant attention from the community for possible innovations in the respective fields.

In the other studied activities, we can see some common patterns that are also important to take into account for improving the support of AI tools. In this regard, among the most promising steps in *Implementing new features*, *Bug triaging* and *Refactoring* are (a) the generation of the new necessary code and (b) the summarization of code for a better understanding of the context of the change. These two activities align with the grounded theory proposed by Barke et al. [17], which describes two developers' interaction modes with AI assistants — acceleration (supported by code generation) and exploration (supported by code summarization). Our detailed results for individual stages can highlight some more specific candidates for focus — *Using code comprehension to summarize the most likely causes of the bug*, *Generating the refactored version of the code*, and others. We suggest that studies of possible improvements in those directions would be important for developers and impactful for the field.

Finally, while the option to chat with the assistant is especially crucial when *implementing new features*, it is present in all the other activities as well. This indicates the importance of not only the "technical" aspects of the models (e.g., generating quality code and summarizing the code correctly) but also UI and UX-related aspects, as mentioned by Zhou et al. [11], specifically conversational functionality of AI assistants as highlighted in works of Ross et al. [33] and Ziegler et al. [14].

These results can be formulated into the following general implications:

- **For researchers.** To bring the most immediate value in the areas where developers already highlight their needs, one can focus on the innovation around the systems that ensure usage of AI assistants in writing tests, writing natural language artifacts, code generation, and code summarization.
- **For tool builders.** The specific features that developers are already actively using — generating tests, summarizing code into docstrings, etc. — are crucial for better integration into the existing tools, such as IDEs. Some of the used features might not yet be directly available in the workflow, such as generating data for the tests, and it is worth experimenting with them.

5.2. What needs to change in the future

Results presented in Section 4.4 indicate a way towards greatly improving the AI experience for users. While many different reasons for not using assistants can be seen in Table 2, still the majority of the respondents said that they are likely or very likely to use AI assistants for all activities if the shortcomings were overcome. The large-scale nature of our survey provides both technical and more fundamental reasons, which makes it difficult to address them equally. For example, *Lack of need for AI assistance* is a more general thing that gradually changes over time. However, the wide array of answers from our respondents, referring to multiple different activities and stages within them, allows us to propose a multi-faceted look at the future research to tackle the existing shortcomings. We see the possible space of innovation threefold: system, integration, and education.

One direction is the improvement of base systems of AI assistants. Addressing *AI-generated output being inaccurate* and the *Lack of compliance with non-functional requirements* is crucial, with 17.7% and 3.2% of all responses mentioning these issues, respectively, which is in line with the study of Liang et al. [12]. Previous research shows that developers spend almost 50% of coding time in interaction with the LLM, with 35% dedicated to double-checking suggestions, since the generated code is limited in meeting both functional and non-functional requirements [16]. Therefore, resolving accuracy issues would also affect the *Time inefficiency* of AI tooling usage, mentioned in 5.5% of all responses.

It is also important to resolve *Lack of understanding context by AI assistants*. This issue was raised in 14.4% of all responses and represents one of the most specific and concrete reasons for not using AI assistants. One part of it is the inability of the models to consider the entire codebase of the project. Another aspect of this issue is the assistant's lack of access to other sources of information, such as project-specific documentation, internal knowledge bases, issue trackers, etc. In this line lies also the *Limitations of knowledge in the AI model* and consequent *Inefficiency for specific tasks*, mentioned in 1.9% and 1.7% of participants' answers, respectively. In this regard, a pivotal direction for future research is finding ways to compress large amounts of information and project-level context into the model [34,35].

Moreover, *Legal and ethical considerations* and *Security concerns* regarding the use of AI assistance, which came up in 3.1% and 2.3% of responses, respectively, could be addressed via system changes [36]. It is promising to investigate local models that can work on the user devices and not send the data over the internet (e.g., federated learning) [37]. This is also important for *Company policies*, *NDA's*, etc., the issues about which were raised in 3.6% of responses.

We believe that if accuracy, understanding, and compliance with other quality-related requirements were fixed, people would also probably gain more trust receiving more useful and meaningful assistance, resolving the other popular reasons not to use AI—*Lack of trust* and *AI-generated output not being useful*—raised in 15.7% and 4.6% of responses respectively and widely discussed in the community [12,38,39].

Another possible direction for research and innovation is integrating AI systems into developers' workflow. We believe that such issues as *Challenges with AI integration and usability in development environments* (raised in 3% of all responses) and *Workflow disruption* (raised in 2.1% of responses) should be addressed by companies who provide AI assistants for the market, considering their hands-on access to the systems and their users for more quick and meaningful research and innovation [40,41].

Moreover, it is important to educate and inform users and potential users about the capabilities and limitations of AI assistants. Mentioned in 10.2% of responses, *User's limited knowledge or understanding of AI assistants and their capabilities*, and in 1% of responses — *Challenges in communicating user intentions to AI assistant*, these drawbacks may be addressed by the wider and specific knowledge-sharing activities in the community and beyond it [42,43]. Moreover, with the evolution of AI-supported educational activities [44–46], the community of AI-aware developers will expand greatly in the near future.

These results can be formulated into the following general implications:

- **For researchers.** With the recent advances in developing AI assistants, a lot of different problems remain, all of which require long-term research. In particular, it is important to focus not only on the systems themselves — addressing common issues like lack of quality, understanding larger context, and ensuring legal and security compliance — but also on the human element. Research should align the system with human needs, understanding the reasons behind certain challenges to effectively resolve them and create more user-centered solutions.
- **For tool builders.** A good AI model in itself is not enough to bring value to the user. It is crucial to develop novel ways of integrating AI assistants into the developers' workflow and invest in better UI/UX. With industrial experience in these kinds of improvements, this can be handled by the development teams of tools such as IDEs.
- **For developers.** While the tools should improve their education of users on AI assistants, it is also on developers to inform themselves. Better knowledge of various features and experimenting with new ones can vastly improve the experience, as noted by some responses to our questions that highlighted the desire of the participants to try out new things that they picked up in our survey.

In essence, the findings we gathered offer valuable direction for both short-term research goals and long-term strategies. While developers have voiced a variety of concerns, we are confident that with the concerted efforts of the research community, these issues can be effectively addressed, ultimately leading to a more robust and beneficial user experience.

6. Threats to validity

The large-scale and general nature of our study introduces several important threats to the validity of our study.

Generalizability. Our results are based on a specific sample of people and might not generalize. In particular, even though the mailing list curated by JetBrains includes users of different IDEs and AI assistants, it can introduce some bias into the results. At the same time, our sample of 481 people is large for studies in our field [47], and it is diverse in terms of programming languages and types of developed software.

Thematic analysis. It is possible that our thematic analysis resulted in missing some themes, which could influence our analysis. To combat this, we carefully followed established practices [27]: two authors came up with codes and themes independently and then reached an agreement in the discussion, and we also removed from the analysis responses that did not clearly formulate their issues.

7. Conclusion

In this paper, we set out to study specific ways in which developers use AI assistants in different stages of the software development life-cycle, as well as the reasons why they do not use them. Our results show that respondents use AI assistants in various activities and various stages within them, but unequally. Developers indicated *Writing tests* and *Writing natural language artifacts* as the least enjoyable activities that they would want to delegate to an AI assistant. Within different activities, developers tend to use assistants for generation and summarization, and employ them for finding places in the code and applying code more rarely. In terms of the reasons for not using AI assistants, our thematic analysis revealed 20 diverse issues, with the main ones being *Lack of need for AI assistance*, *AI-generated output being inaccurate*, *Lack of trust*, and *Lack of understanding of context by AI assistant*. We also highlight issues raised by the participants in regard to individual activities, which can inform further research.

We believe our work is especially needed right now, when AI assistance is being developed rapidly, since our comprehensive and specific results can be used to guide further research and implementation.

CRediT authorship contribution statement

Agnia Sergeyyuk: Writing – review & editing, Writing – original draft, Validation, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Yaroslav Golubev:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation. **Timofey Bryksin:** Writing – review & editing, Project administration, Methodology, Conceptualization. **Iftekhar Ahmed:** Writing – review & editing, Project administration, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We would like to extend our sincere gratitude to the Surveys Team and the Research Core Team of JetBrains for their invaluable support and assistance throughout this study. Their expertise and dedication were crucial in facilitating the research process and ensuring its success. We greatly appreciate their contributions and collaboration.

Data availability

The detailed description of the survey, the collected responses, as well as some additional results that did not fully fit into the paper can be found in supplementary materials [21].

References

- [1] J. Kaddour, J. Harris, M. Mozes, H. Bradley, R. Raileanu, R. McHardy, Challenges and applications of large language models, 2023, arXiv preprint [arXiv:2307.10169](https://arxiv.org/abs/2307.10169).
- [2] GitHub Copilot, 2024, <https://github.com/features/copilot>. (Accessed November 2024).
- [3] JetBrains AI, 2024, <https://www.jetbrains.com/ai/>. (Accessed November 2024).
- [4] Visual Studio IntelliCode, 2024, <https://visualstudio.microsoft.com/services/intellicode/>. (Accessed November 2024).
- [5] OpenAI, OpenAI Codex, 2021, <https://openai.com/blog/openai-codex>. (Accessed November 2024).
- [6] B. Berabi, A. Gronskiy, V. Raychev, G. Sivanrupan, V. Chibotaru, M. Vechev, DeepCode AI fix: Fixing security vulnerabilities with large language models, 2024, arXiv preprint [arXiv:2402.13291](https://arxiv.org/abs/2402.13291).
- [7] CodeScene, 2024, <https://codescene.com/>. (Accessed November 2024).
- [8] Testsigma AI-driven test automation, 2024, <https://testsigma.com/ai-driven-test-automation>. (Accessed November 2024).
- [9] Developer sentiment around AI/ML, 2023, <https://stackoverflow.co/labs/developer-sentiment-ai-ml>. (Accessed November 2024).
- [10] C. Wang, J. Hu, C. Gao, Y. Jin, T. Xie, H. Huang, Z. Lei, Y. Deng, Practitioners' expectations on code completion, 2023, arXiv preprint [arXiv:2301.03846](https://arxiv.org/abs/2301.03846).
- [11] X. Zhou, P. Liang, B. Zhang, Z. Li, A. Ahmad, M. Shahin, M. Waseem, Exploring the problems, their causes and solutions of AI pair programming: a study on GitHub and Stack Overflow, *Journal of Systems and Software* 219 (2025) 112204.
- [12] J.T. Liang, C. Yang, B.A. Myers, A large-scale survey on the usability of AI programming assistants: Successes and challenges, in: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.
- [13] C. Wang, J. Hu, C. Gao, Y. Jin, T. Xie, H. Huang, Z. Lei, Y. Deng, How practitioners expect code completion? in: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1294–1306.
- [14] A. Ziegler, E. Kalliamvakou, X.A. Li, A. Rice, D. Rifkin, S. Simister, G. Sittampalam, E. Aftandilian, Productivity assessment of neural code completion, in: *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, 2022, pp. 21–29.
- [15] A.S. Pothukuchi, L.V. Kota, V. Mallikarjunaradhya, Impact of generative AI on the software development lifecycle (SDLC), *Int. J. Creat. Res. Thoughts* 11 (8) (2023).
- [16] H. Mozannar, G. Bansal, A. Fourney, E. Horvitz, Reading between the lines: Modeling user behavior and costs in AI-assisted programming, in: *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–16.
- [17] S. Barke, M.B. James, N. Polikarpova, Grounded Copilot: How programmers interact with code-generating models, *Proc. ACM Program. Lang.* 7 (OOPSLA1) (2023) 85–111.
- [18] Stack Overflow Knows code assistant pulse survey results, 2024, <https://stackoverflow.blog/2024/05/29/developers-get-by-with-a-little-help-from-ai-stack-overflow-knows-code-assistant-pulse-survey-results/>. (Accessed November 2024).
- [19] GitHub, Survey reveals AI's impact on the developer experience, 2024, <https://github.blog/2023-06-13-survey-reveals-ais-impact-on-the-developer-experience/>. (Accessed November 2024).
- [20] JetBrains developer ecosystem survey 2023 on AI, 2024, <https://www.jetbrains.com/lp/devecosystem-2023/ai/>. (Accessed November 2024).
- [21] A. Sergeyuk, Y. Golubev, T. Bryksin, I. Ahmed, Supplementary materials, 2024, <https://doi.org/10.5281/zenodo.10854382>. (Accessed November 2024).
- [22] M.M. Lehman, Programs, life cycles, and laws of software evolution, *Proc. IEEE* 68 (9) (1980) 1060–1076.
- [23] ICC/ESOMAR international code on market, opinion and social research and data analytics, 2016, <https://esomar.org/uploads/attachments/ckqtawvj00uukdrthst5sk9u-iccesomar-international-code-english.pdf>. (Accessed November 2024).
- [24] A.W. Meade, S.B. Craig, Identifying careless responses in survey data, *Psychol. Methods* 17 (3) (2012) 437.
- [25] W.P. Zijlstra, L.A. Van Der Ark, K. Sijtsma, Outlier detection in test and questionnaire data, *Multivar. Behav. Res.* 42 (3) (2007) 531–555.
- [26] J. Murray, Likert data: what to use, parametric or non-parametric? *Int. J. Bus. Soc. Sci.* 4 (11) (2013).
- [27] J. Fereday, E. Muir-Cochrane, Demonstrating rigor using thematic analysis: A hybrid approach of inductive and deductive coding and theme development, *Int. J. Qual. Methods* 5 (1) (2006) 80–92.
- [28] J. Forman, L. Damschroder, Qualitative content analysis, in: *Empirical Methods for Bioethics: A Primer*, Emerald Group Publishing Limited, 2007, pp. 39–62.
- [29] V. Braun, V. Clarke, Using thematic analysis in psychology, *Qual. Res. Psychol.* 3 (2) (2006) 77–101.
- [30] G. Sandoval, H. Pearce, T. Nys, R. Karri, S. Garg, B. Dolan-Gavitt, Lost at C: A user study on the security implications of large language model code assistants, in: 32nd USENIX Security Symposium (USENIX Security 23), 2023, pp. 2205–2222.
- [31] O. Asare, M. Nagappan, N. Asokan, Is GitHub's Copilot as bad as humans at introducing vulnerabilities in code? *Empir. Softw. Eng.* 28 (6) (2023) 129.
- [32] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, R. Karri, Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions, in: 2022 IEEE Symposium on Security and Privacy, SP, 2022, pp. 754–768, <http://dx.doi.org/10.1109/SP46214.2022.9833571>.
- [33] S.I. Ross, F. Martinez, S. Houde, M. Muller, J.D. Weisz, The programmer's assistant: Conversational interaction with a large language model for software development, in: *Proceedings of the 28th International Conference on Intelligent User Interfaces*, 2023, pp. 491–514.
- [34] A. Lozhkov, R. Li, L.B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei, et al., StarCoder 2 and The Stack v2: The next generation, 2024, arXiv preprint [arXiv:2402.19173](https://arxiv.org/abs/2402.19173).
- [35] CrossCodeEval, 2023, <https://crosscodeeval.github.io/>. (Accessed November 2024).
- [36] OpenAI lawsuit, 2022, <https://githubcopilotlitigation.com/>. (Accessed November 2024).
- [37] L. Li, Y. Fan, M. Tse, K.-Y. Lin, A review of applications in federated learning, *Comput. Ind. Eng.* 149 (2020) 106854.
- [38] Communication: Building trust in human-centric artificial intelligence, 2019, <https://digital-strategy.ec.europa.eu/en/library/communication-building-trust-human-centric-artificial-intelligence>. (Accessed November 2024).
- [39] M. Amoozadeh, D. Daniels, D. Nam, A. Kumar, S. Chen, M. Hilton, S. Srinivasa Ragavan, M.A. Alipour, Trust in generative AI among students: An exploratory study, in: *Proceedings of the 55th ACM Technical Symposium on Computer Science Education*, Vol. 1, 2024, pp. 67–73.
- [40] P. Vaithilingam, E.L. Glassman, P. Groenewegen, S. Gulwani, A.Z. Henley, R. Malpani, D. Pugh, A. Radhakrishna, G. Soares, J. Wang, et al., Towards more effective AI-assisted programming: A systematic design exploration to improve Visual Studio IntelliCode's user experience, in: 2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), IEEE, 2023, pp. 185–195.
- [41] H. Mozannar, G. Bansal, A. Fourney, E. Horvitz, When to show a suggestion? Integrating human feedback in AI-assisted programming, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38, 2024, pp. 10137–10144.
- [42] L. Feng, R. Yen, Y. You, M. Fan, J. Zhao, Z. Lu, CoPrompt: supporting prompt sharing and referring in collaborative natural language programming, in: *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–21.
- [43] ACM CUI, CHI 2024 workshop, 2024, <https://cui.acm.org/workshops/CHI2024/>. (Accessed November 2024).
- [44] J. Chen, X. Lu, Y. Du, M. Rejtig, R. Bagley, M. Horn, U. Wilensky, Learning agent-based modeling with LLM companions: Experiences of novices and experts using ChatGPT & NetLogo chat, in: *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–18.
- [45] P. Robe, S.K. Kuttal, Designing PairBuddy—A conversational agent for pair programming, *ACM Trans. Comput.-Hum. Interact.* 29 (4) (2022) 1–44.
- [46] D. Jayagopal, J. Lubin, S.E. Chasins, Exploring the learnability of program synthesizers by novice programmers, in: *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology*, 2022, pp. 1–15.
- [47] F. Medeiros, M. Ribeiro, R. Gheyi, S. Apel, K. Kästner, B. Ferreira, L. Carvalho, B. Fonseca, Discipline matters: Refactoring of preprocessor directives in the #ifdef hell, *IEEE Trans. Softw. Eng.* 44 (5) (2017) 453–469.